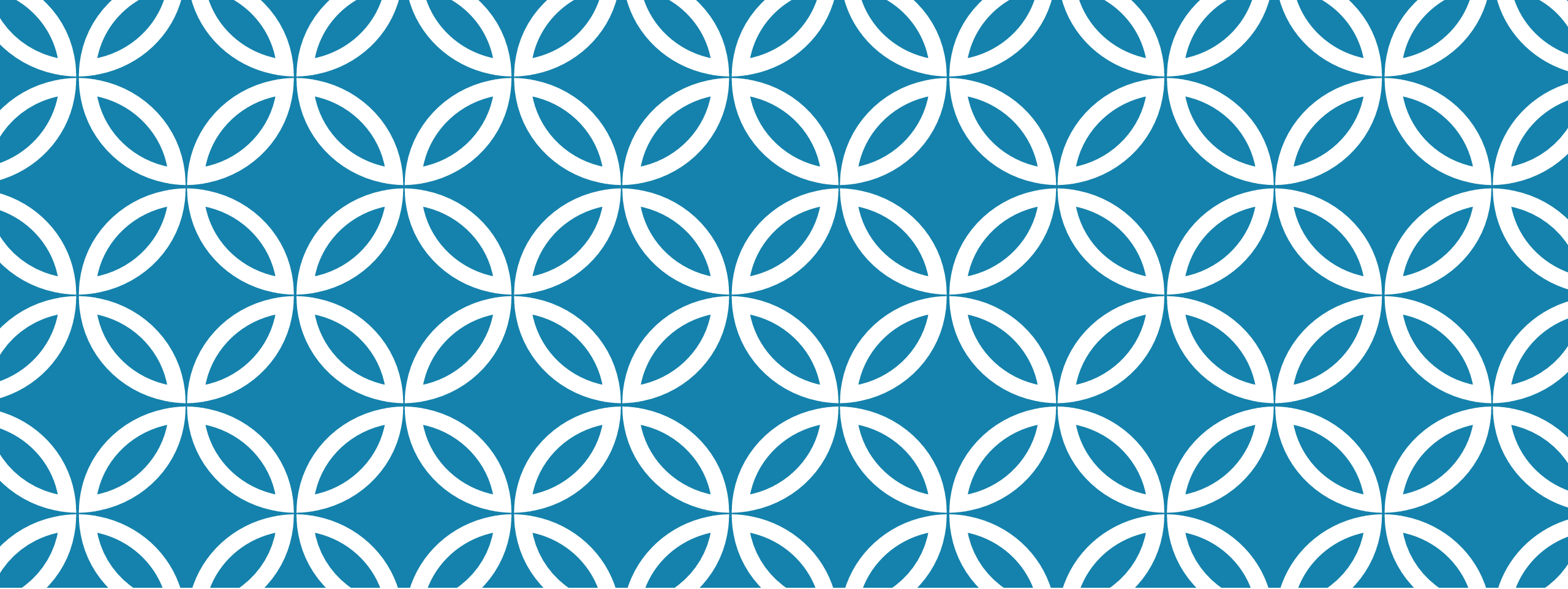




DOM 基础与框架认知

从命令式操作到声明式渲染

目标：用两种方案写出同一个计数器，并
说清它们的本质差异

目录：四个部分

- 一、DOM 核心概念
 - 节点树 · 节点关系 · 元素对象与 HTML 属性的映射
- 二、原生 DOM 编程接口
 - 查询 · 内容 · 类名 · 样式 · 事件 · 创建与挂载
- 三、声明式 UI 编程 (Vue3 CDN)
 - createApp · ref / reactive · 模板语法 · 指令
- 四、阶段实战 —— 两种计数器
 - 命令式三步 vs 声明式数据驱动

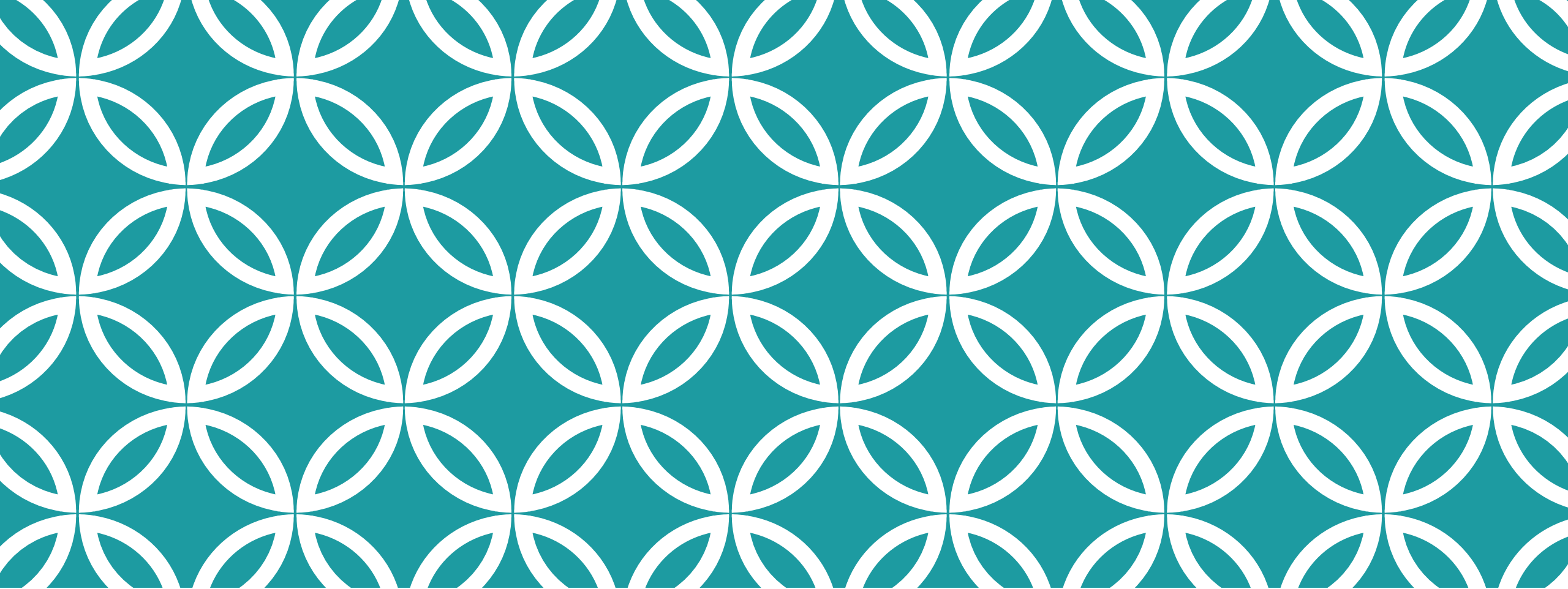
学完你能做出什么

- 一个页面，左右两个计数器 —— 功能完全一样，写法天差地别
- 左边原生 DOM：查询元素 → 改内容 → 绑事件，每一步都要你亲手写
- 右边 Vue3：只定义 `count = ref(0)`，改数字，页面自己更新
- 明确：命令式三步 与 声明式数据驱动 的本质差异

原生 DOM
`getElementById`
`textContent`
`addEventListener`

Vue3 CDN
`createApp + setup`
`ref(0)`
`{{ count }}` `@click`

改数字，别的都不用管 →



一、DOM 核心概念

目标：页面在浏览器里到底长成什么样

什么是 DOM

- DOM = Document Object Model, 文档对象模型
- 浏览器把 HTML 解析成一棵对象树, 每个标签都是一个对象
- JS 通过操作这棵树, 就能改变页面的内容与结构
- document 是整棵树的入口, 也是所有查询的起点
- DOM 不是 HTML 源码本身, 而是浏览器解析后的运行时对象

一句话: HTML 是文本, DOM 是浏览器把它变成的对象

在 DevTools 里看 DOM 树

- 打开：F12 / Ctrl+Shift+I → Elements 面板
- 点左上角箭头（或 Ctrl+Shift+C）再点页面元素，自动定位到对应节点
- 展开 / 折叠三角箭头看子节点；鼠标悬停时页面会高亮对应区域
- 选中节点后在 Console 输入 \$0，就是这个元素对象
- ⚠ 面板里双击改的是 DOM 不是源码，刷新即还原

```
// 1) 选中页面上的按钮后，在 Console 里：  
$0 // → <button id="add">+1</button>  
$0.textContent // → '+1'  
$0.id // → 'add'  
$0.classList // → DOMTokenList ['btn']  
  
// 2) 也可以直接查询  
document.querySelector('#add')
```

节点关系：父子与兄弟

- 父：parentNode（或 parentElement）
- 子：children（只含元素） / childNodes（含文本节点） / firstElementChild
- 兄：previousElementSibling / nextElementSibling
- ⚠ 想遍历元素就用 children，用 childNodes 会把换行文本也算进去

```
<ul id="list">
  <li>苹果</li>
  <li>香蕉</li>
  <li>橙子</li>
</ul>
```

以上含3个children，7个childNodes

```
文本节点(\n )
元素节点 <li>苹果</li>
文本节点(\n )
元素节点 <li>香蕉</li>
文本节点(\n )
元素节点 <li>橘子</li>
文本节点(\n )
```

```
<!DOCTYPE html>
<html lang="zh">
<head>
  <meta charset="UTF-8">
  <title>DOM 测试</title>
</head>
<body>
  <ul id="list">
    <li>苹果</li>
    <li>香蕉</li>
    <li>橘子</li>
  </ul>

  <script>
    const el = document.getElementById('list');
    console.log(el.children.length); // 3
    console.log(el.childNodes.length); // 7
    console.log(el.children[0].textContent); // 苹果
    console.log(list.children[1].nextElementSibling.textContent )
// '橘子'
    console.log(list.children[0].parentNode === list) //
true
    console.log(list.firstElementChild); // <li>苹果</li>
  </script>
</body>
</html>
```

DOM 树的四类节点

- 文档节点 document: 整棵树的根, 全局唯一
- 元素节点 element: 每个标签, 如 div / p / img
- 文本节点 text: 标签里的文字 (换行和空格也算)
- 属性节点 attribute: 标签上的 id / class / src

```
<div id="box" class="card">Hello</div>
```

```
document
├─ html
│   └─ body
│       └─ div#box           ← 元素节点
│           ├── "Hello"      ← 文本节点
│           ├── id="box"     ← 属性节点
│           └─ class="card"  ← 属性节点
```

节点遍历实操

- 思路：先定位一个已知节点，再顺着关系链往上、往下、往旁边走
- 遍历时优先用 Element 系列（children / nextElementSibling），跳过文本节点

```
const li = document.querySelector('#list li'); // 第一个 li
console.log(li.textContent);                    // 苹果

// 往旁边走
li.nextElementSibling.textContent;             // 香蕉
li.nextElementSibling.nextElementSibling.textContent; // 橙子

// 往上走再往下走
li.parentNode.children.length;                // 3
li.parentNode.lastElementChild.textContent;    // 橙子

// 遍历全部
[...li.parentNode.children].forEach((el) => console.log(el.textContent));
```

元素对象 ↔ HTML 属性的对应关系

HTML 写法	DOM 对象属性	取值示例
<code>id="pic"</code>	<code>el.id</code>	"pic"
<code>class="thumb"</code>	<code>el.className</code>	"thumb"
<code>src="a.png"</code>	<code>el.src</code>	完整 URL 而非相对路径
<code>href="#top"</code>	<code>el.href</code>	完整 URL
<code>value="提交"</code>	<code>el.value</code>	"提交" (当前值)
<code>title="提示"</code>	<code>el.title</code>	"提示"
<code>style="color:red"</code>	<code>el.style.color</code>	"red"
(无对应属性)	<code>el.classList</code>	DOMTokenList, 用 add/remove 操作

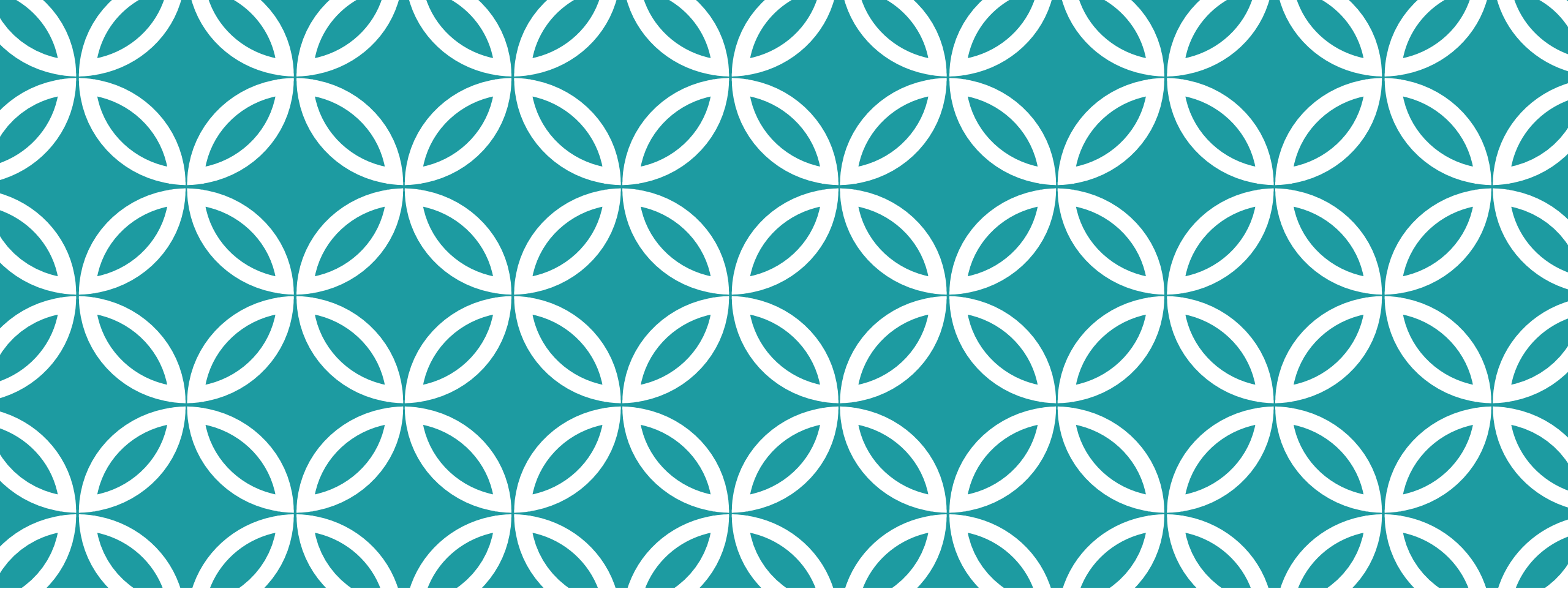


易混：HTML 特性 vs DOM 属性

- 特性 attribute：写在标签里的，用 `getAttribute` 读，拿到的是初始值
- 属性 property：对象上的属性，直接点出来，拿到的是当前值
- `class` 是特性，对应的属性叫 `className`
- 用户在输入框里打字后：attribute 不变，property 跟着变
- 读当前状态用 property；读初始配置用 attribute

```
<input id="kw" value="初始">
const kw = document.getElementById('kw');
kw.getAttribute('value')    // '初始'    标签里写死的
kw.value                   // '初始'    当前值

// 用户输入「JavaScript」之后：
kw.getAttribute('value')    // '初始'    ← 没变！
kw.value                   // 'JavaScript' ← 变了
```



二、原生 DOM 编程接口

目标：查询 → 修改 → 绑定，独立完成命令式操作

元素查询（一）：getElement 系列

- getElementById('id') → 单个元素，找不到返回 null
- getElementsByClassName('cls') → HTMLCollection
- getElementsByTagName('li') → HTMLCollection
- ⚠ 返回的是动态集合：DOM 一变，它跟着变

```
<ul id="list">
  <li class="item">苹果</li>
  <li class="item">香蕉</li>
  <li class="item">橘子</li>
</ul>
```

```
document.getElementById('count')      // <span id="count">0</span>
document.getElementsByClassName('item') // HTMLCollection(3), 类似数组
document.getElementsByTagName('li')     // HTMLCollection(3)

const items = document.getElementsByClassName('item');
console.log(items.length);              // 3
document.body.appendChild(newItem);
console.log(items.length);              // 4 ← 动态集合会变

// HTMLCollection 没有 map/filter, 转数组再用:
[...items].map((el) => el.textContent)
```

元素查询（二）：querySelector 系列

- `querySelector('选择器')` → 第一个匹配的元素，找不到返回 `null`
- `querySelectorAll('选择器')` → 所有匹配的 `NodeList`（静态）
- 支持完整 CSS 选择器：`#id` `.class` `[attr]` 后代 子代 伪类
- `NodeList` 没有 `map` / `filter`，用之前先转数组 `[...list]`

```
document.querySelector('#app')           // id
document.querySelector('.item.active')    // 选中同时拥有 item 和 active 两个 class 的元素。
（此例子中间没有空格；如果有空格，表示item 元素内部的所有的active后代元素）
document.querySelector('ul > li:first-child')
// >取第一层元素（子不含孙），取第一个li。
document.querySelectorAll('#list li')     // NodeList(3) 静态

const lis = document.querySelectorAll('#list li');
lis.map(...)                             // ✗ TypeError: NodeList 没有 map
[...lis].map((el) => el.textContent)      // ✓ 先转数组
```

内容操作：textContent 与 innerHTML

- textContent：读写纯文本，标签会被当成普通字符显示（安全）
- innerHTML：读写 HTML，内容会被解析成真实标签，解析并执行 HTML 是它的能力，也是它的危险来源。
- innerHTML 把字符串当 HTML 解析。如果字符串里混进了用户输入，而用户输入里带了 <script> 或 onerror 之类的东西，浏览器就会真的执行它。这就造成 XSS（跨站脚本攻击）。
- 只要显示文字，一律用 textContent

```
el.textContent = '<b>你好</b>';  
// 页面显示：<b>你好</b>          当作文字  
  
el.innerHTML = '<b>你好</b>';  
// 页面显示：你好（加粗）          解析成标签  
  
// ✕ 危险：用户输入被当成 HTML 执行  
el.innerHTML = userInput; // 用户可插入 <script>  
// ☑ 安全  
el.textContent = userInput;  
  
console.log(el.textContent); // 读：拿到纯文本
```

类名操作：classList

- class List是元素上专门操作 class 列表的接口。它把 class 当多个独立的类来处理，而 className 把 class 当一整个字符串处理。
- add('a') remove('a') toggle('a') contains('a')
- toggle: 有则删、无则加 —— 做开关状态最合适; 一次操作多个: add('a', 'b')
- ⚠ 用 className = "x" 会整体覆盖掉原有类名,className 获取的是一整个字符串
- 完整验证代码见下页

```
<!DOCTYPE html>
<html lang="zh">
<head>
  <meta charset="UTF-8">
  <title>classList 示例</title>
  <style>
    .btn { padding: 8px 16px; border: 1px
solid #999; }
    .active { background: #4caf50; color:
white; }
  </style>
</head>
<body>
  <button class="btn">按钮</button>
  <script>
    代码见右
  </script>
</body>
</html>
```

```
const btn = document.querySelector('.btn');

console.log(btn.className);           // "btn"
console.log(btn.classList);          // DOMTokenList ["btn"]

btn.classList.add('active');
console.log(btn.className);           // "btn active"

btn.classList.remove('active');
console.log(btn.className);           // "btn"

btn.classList.toggle('active');
console.log(btn.className);           // "btn active"

console.log(btn.classList.contains('active')); // true

btn.classList.add('a', 'b');
console.log(btn.className);           // "btn active a b"

// 整体覆盖，原来的全没了
btn.className = 'x';
console.log(btn.className);           // "x"
```

样式操作：element.style

- 驼峰命名：background-color → backgroundColor
- 只能读写行内样式（style 属性里的），读不到 <style> 里写的
- window.getComputedStyle(element) 是浏览器提供的函数，返回该元素最终生效的、计算后的样式。
- 批量改样式优先切 classList，别一个个设 style

```
const box = document.querySelector('#box');
box.style.backgroundColor = '#1CADE4'; // ☑ 驼峰
box.style['background-color'] = '#1CADE4'; // 也可用方括号
box.style.fontSize = '18px';
box.style.backgroundColor // '#1CADE4' 读得到

// ✕ 读不到 CSS 文件里写的样式
box.style.margin // '' 空字符串
getComputedStyle(box).margin // '0px' ☑

// 批量改样式：切类名更清爽
box.classList.add('highlight');
```

属性操作: get / set / remove / dataset

- `getAttribute(name)` `setAttribute(name, value)` `removeAttribute(name)`
- `hasAttribute(name)` 判断是否存在
- `data-*` 自定义属性 → `el.dataset.xxx`, 多段用驼峰
- `data-user-id` → `el.dataset.userId`

```

```

```
const pic = document.getElementById('pic');  
pic.getAttribute('src')           // 'a.png' 拿原始写法  
pic.src                          // 完整 URL  
pic.setAttribute('alt', '封面');  
pic.removeAttribute('alt');
```

```
pic.dataset.id                   // '7'  
pic.dataset.userId               // '42' 驼峰
```

```
pic.hasAttribute('data-id')      // true
```

事件绑定：addEventListener

- 语法：el.addEventListener('click', 回调函数)
- ⚠ 事件名不带 on：写 'click' 不是 'onclick'
- 回调函数的第一个参数就是事件对象 e
- 同一元素可绑多个同类型事件，按绑定顺序依次执行
- 移除用 removeEventListener，必须传同一个函数引用

```
const btn = document.querySelector('#add');
function onClick(e) {
  console.log('被点了', e);      // e 是事件对象
}
btn.addEventListener('click', onClick);
btn.addEventListener('click', () => console.log('第二个'));
btn.removeEventListener('click', onClick);  // 同一个引用才行

// ❌ 常见错误
btn.addEventListener('onclick', fn);  // 多了 on, 永远不触发
```

事件对象 e 与事件传播

- e.target: 真正触发事件的元素（鼠标点到的那个）
- e.currentTarget: 绑定事件的元素（回调里的 this）
- 三阶段：捕获（外→内）→ 目标 → 冒泡（内→外）
- e.preventDefault() 阻止默认行为（表单提交、链接跳转）
- e.stopPropagation() 阻止继续向上传播

```
<div id="outer"><button id="inner">点我</button></div>
document.querySelector('#outer').addEventListener('click', (e) => {
  console.log('target:', e.target.id);          // inner
  console.log('currentTarget:', e.currentTarget.id); // outer
});

// 阻止默认行为
form.addEventListener('submit', (e) => {
  e.preventDefault();      // 不刷新页面
});
```

事件委托

- 问题：给 li 逐个绑事件，后添加的 li 不会响应
- 做法：把事件绑在父元素上，靠冒泡统一处理
- 判断来源：e.target.matches("li") 或 e.target.closest("li")
- 好处：只绑一次、新元素自动生效、内存占用小

```
// ✗ 逐个绑定：新增的 li 没有事件
document.querySelectorAll('li').forEach((li) =>
  li.addEventListener('click', () => console.log(li.textContent)));

// ☑ 事件委托：绑在父元素，一次搞定
document.querySelector('#list').addEventListener('click', (e) => {
  if (!e.target.matches('li')) return; // 只处理 li
  console.log('点击了：', e.target.textContent);
});

// 之后 appendChild 进来的新 li，一样能响应
```

常见事件速查表

事件名	触发时机	常用元素
<code>click / dblick</code>	鼠标点击 / 双击	所有元素
<code>input</code>	输入框内容变化（实时）	input、textarea
<code>change</code>	值改变且失去焦点	select、input[type=file]
<code>submit</code>	表单提交	form
<code>focus / blur</code>	获得 / 失去焦点	表单控件
<code>keydown / keyup</code>	键盘按下 / 松开	input、document
<code>mouseenter / mouseleave</code>	鼠标移入 / 移出（不冒泡）	所有元素
<code>scroll</code>	元素或页面滚动	window、滚动容器

input 实时、change 失焦 —— 做搜索框用 input

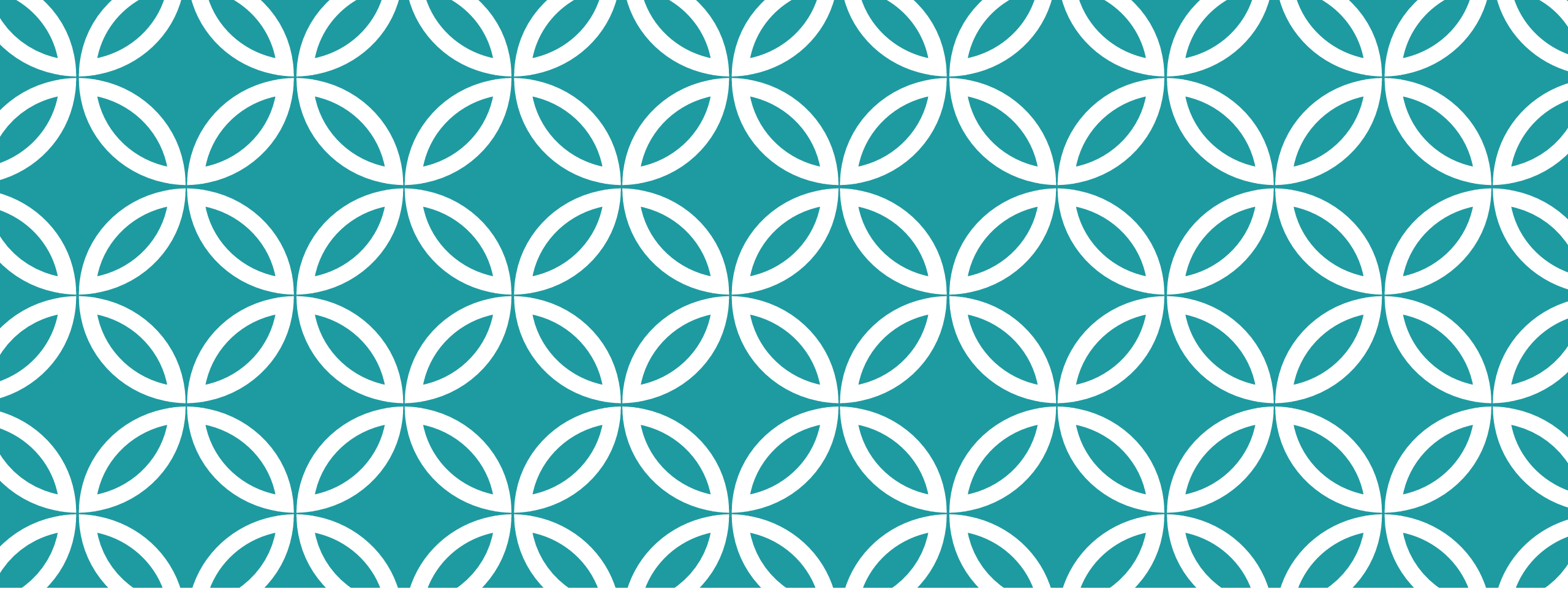
元素创建与挂载

- createElement('li') 创建 —— 此时还在内存里，页面上没有
- 设置内容与类名：li.textContent / li.classList.add
- 挂载：parent.appendChild(li) 加到末尾
- 插到前面 insertBefore(新, 参考节点); 删除 el.remove()

```
const list = document.querySelector('#list');
// 1. 创建（内存中）
const li = document.createElement('li');
// 2. 设置
li.textContent = '新项目';
li.classList.add('item');

// 3. 挂载（出现在页面上）
list.appendChild(li);           // 加到末尾
list.insertBefore(li, list.firstChild); // 插到最前

li.remove();                    // 删除自己
```



三、声明式 UI 编程 (VUE3 CDN)

目标：只改状态，视图自动更新的计数器

表单取值专题

- 文本框 / 文本域: `el.value` —— 拿到的永远是字符串
- 要当数字用必须转: `Number(el.value)` 或 `parseInt`
- 复选框: `el.checked` (`true / false`, 不是 `value`)
- 下拉框: `select.value`; 单选: 找 `checked` 的那一个

```
const age = document.querySelector('#age');
age.value           // '18'   字符串!
Number(age.value) + 1 // 19    ☒ 先转数字
age.value + 1       // '181'  ☒ 字符串拼接

const agree = document.querySelector('#agree');
agree.checked    // true / false

document.querySelector('#city').value // 下拉选中项的 value
document.querySelector('input[name=sex]:checked').value // 单选
```

value 永远是字符串, 做算术前先 `Number()`

小结：命令式三步

- 第一步 查询元素： `const el = getElementById(...)`
- 第二步 修改内容： `el.textContent = count`
- 第三步 绑定事件： `btn.addEventListener(...)`
- 每加一个功能，这三步就要再写一遍
- 状态在变量里、视图在 DOM 里，靠人手同步 —— 漏一处就显示错

如果状态和视图能自动同步，这三步是不是就不用写了？

为什么需要框架

- 原生方案的状态散落各处：一半在变量里，一半在 DOM 里
- 每次状态变化都要记得手动更新 DOM，漏一处就显示错
- 列表增删要重建整段 innerHTML，或逐个 createElement
- 代码一多，同步逻辑会淹没真正的业务意图
- 框架的思路：你只管改状态，视图更新交给框架

框架不是让 DOM 操作消失，而是把它封装起来不用你写

环境搭建：CDN 零构建（了解即可）

- 一个 script 标签引入 Vue3，不需要 Node、不需要打包工具
- 引入后得到全局对象 Vue，API 都挂在它上面（Vue.createApp、Vue.ref）
- 需要一个挂载点：<div id="app"></div>
- ⚠ script 必须写在挂载点之后，否则取不到 #app

- vue.global.js 是一个打包好的全局 Vue 对象
- 一个运行时（runtime）：负责响应式、渲染、更新
- 一个编译器（compiler）：把模板编译成渲染函数
- 一套约定/API：createApp、ref、computed、watch……

```
<!DOCTYPE html>
<html>
<body>
  <div id="app">{{ count }}<button @click="count++">加一</button></div>

  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  <script>
    const { createApp } = Vue; //es6的解构赋值，相当于const createApp = Vue.createApp;
    //可以多个API一起解构  const { createApp, ref, computed } = Vue;后面就不要写Vue.createApp，直接createApp即可
    createApp({      //创建 Vue 应用实例
      data() {
        return { count: 0 };
      }
    }).mount('#app');    // 一个 Vue 应用只有一个根挂载点。其他组件嵌套在根组件内部。挂载时，Vue 会编译并渲染模板。
  </script>
</body>
</html>
```

创建应用：createApp + mount

- `Vue.createApp({ setup() { ... } }).mount('#app')`
- `setup()` 是组合式 API 的入口，它返回的对象会暴露（export）给模板（Template，HTML部分）
- `mount` 把应用挂到挂载点，挂载点内部就变成模板
- 挂载后 `#app` 由 Vue 接管，不要再手动改它的 DOM

```
const { createApp, ref } = Vue;    // 从全局对象上取

createApp({
  setup() {
    const count = ref(0);           // 状态
    const increment = () => { count.value++; };
    return { count, increment };    // 暴露给模板
  }
}).mount('#app');                  // 挂到 #app

// 模板里就能直接用 count 和 increment
```

响应式状态（一）：ref

- `const count = ref(0)` 定义一个响应式状态
- ⚠ 脚本（setup）里必须写 `count.value`
- 模板里自动解包，直接写 `{{ count }}`，不用 `.value`
- `ref` 可以包任何类型，基本类型就用它

```
setup() {  
  const count = ref(0);  
  console.log(count);           // RefImpl 对象，不是数字  
  console.log(count.value);     // 0      脚本里要 .value  
  
  const increment = () => {  
    count.value++;              // ☑ 改 .value 才会触发更新  
  };  
  
  return { count, increment };  
}  
// 模板: <span>{{ count }}</span>    模板里不用 .value
```

`ref(0) : count`——带响应式能力的包装对象

1. 创建一个响应式引用，把 0 包在里面
2. 返回一个对象（`RefImpl`这个类的实例），结构类似：

```
{ value:0,           // 实际值  
  __v_isRef:true     // 标记它是 ref }
```

3. `{{ count }}`（自动解包）Vue 在编译模板时，会对 `ref` 做特殊处理，`{{ count }}` 会自动编译为 js 语句 `toDisplayString(count.value)`
4. `ref` 是对象，`const` 不影响改 `.value`，可防止把变量重新指向。

响应式状态（二）：reactive

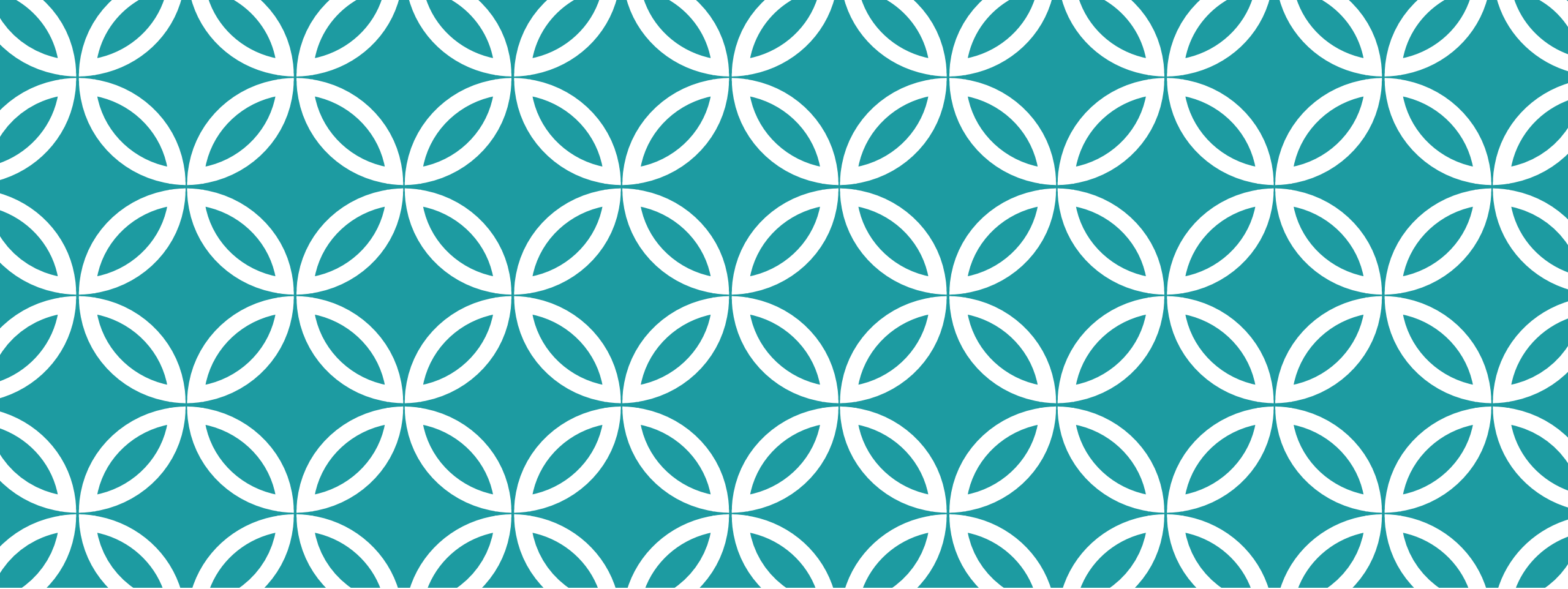
- `reactive(obj)` 把普通对象变成一个“被监控的”对象——读写它的任何属性时，Vue 都能感知到。使得数据被改变时，页面就能自动更新。这个“被监控的对象”被称为响应式代理对象（Proxy）
- `reactive` 用于对象 / 数组：`reactive({ n: 0, list: [] })`
- 访问时不需要 `.value`：`state.n++`
- ⚠ 解构会丢响应式：`const { n } = state` 之后 `n` 不再响应
- 口诀：基本类型用 `ref`，对象数组用 `reactive`

```
<!DOCTYPE html>
<html>
<body>
  <div id="app">
    <p>{{ state.n }}</p>
    <button @click="state.n++">加一</button>
  </div>

  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  <script>
    const { createApp, reactive } = Vue;
    createApp({
      setup() {
        const state = reactive({ n: 0 });
        return { state };
      }
    }).mount('#app');
  </script>
</body>
</html>
```

运行流程：

- 1.首次渲染：Vue 读 `state.n` → 触发 `get` → `track` 记录“渲染函数用了 `state.n`”
- 2.点击按钮：`state.n++` → 触发 `set` → `trigger` 找到“用到 `state.n` 的渲染函数”
- 3.重新渲染：渲染函数重新执行，`{{ state.n }}` 变成新值



四、阶段实战 —— 两种计数器

独立完成：同一功能，两种写法，并说清本质差异

任务总览与要求（不用完成）

- 任务：创建两个独立 HTML 页面（或同一页面两个独立区域），实现同一计数器
- 功能：显示当前数字，点击按钮 +1
 - 方案一 原生： `getElementById + textContent + addEventListener`
 - 方案二 Vue3： `createApp + setup + ref(0) + {{ count }} + @click`
- 完成后须能阐述两者的本质差异：
 - 命令式三步（查询 → 修改 → 绑定） vs 声明式数据驱动（改状态自动更新视图）
- 交付：可运行的页面 + 一段差异说明

方案一：原生 DOM 实现

- ① 查询元素 → ② 修改内容 → ③ 绑定事件，一步都不能少

```
<span id="count">0</span>
<button id="add">+1</button>

<script>
  // ① 查询元素
  const span = document.getElementById('count');
  const btn  = document.getElementById('add');

  let count = 0;                // 状态：普通变量

  // ③ 绑定事件
  btn.addEventListener('click', () => {
    count++;
    // ② 手动修改内容（忘写这行页面就不动）
    span.textContent = count;
  });
</script>
```

方案二：Vue3 CDN 实现

- ① 定义状态 → ② 模板渲染 → ③ 绑定事件；改状态，视图自动更新

```
<div id="app">
  <!-- ② 模板渲染 -->
  <span>{{ count }}</span>
  <!-- ③ 绑定事件 -->
  <button @click="increment">+1</button>
</div>

<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
<script>
  Vue.createApp({
    setup() {
      // ① 定义响应式状态
      const count = Vue.ref(0);
      const increment = () => { count.value++; };
      return { count, increment }; // 没有一行代码碰 DOM
    }
  }).mount('#app');
</script>
```

本质差异：命令式 vs 声明式

维度	方案一 原生 DOM	方案二 Vue3
获取元素	getElementById 手动取	不需要取
修改内容	span.textContent = count	模板 {{ count }}
绑定事件	addEventListener('click', fn)	@click="increment"
状态存放	普通变量 let count = 0	响应式 ref(0)
视图更新	手动调用，忘写就不同步	状态变，自动更新
加一个功能	查询+修改+绑定 再写一遍	加一个方法即可
编程范式	命令式：一步步指挥怎么做	声明式：描述结果是什么

深入理解：DOM 操作被封装与替代

- 我们只改了 `count.value`，页面上的数字就变了 —— 中间发生了什么？
- ① 响应式系统：状态被读取时记录依赖，被修改时通知依赖
- ② 重新渲染：生成新的虚拟 DOM（描述页面应该长什么样）
- ③ 比对：与旧的虚拟 DOM 比较，找出真正变化的最小部分
- ④ 打补丁：只把那一点点变化更新到真实 DOM



DOM 操作没有消失，只是被框架收进了响应式系统与渲染器

扩展练习（挑战）

- ① 加"减 1"按钮 —— 数一数两种方案各改了几处？
- ② 加"重置"按钮，数字归零
- ③ 加步长：点一次加 step，step 由输入框决定（v-model.number）
- ④ 挑战：显示"当前是奇数 / 偶数"（用 computed）
- 体会：功能越复杂，两种方案的代码量差距越大

```
// Vue 版扩展：加一个 computed 就够了
setup() {
  const count = Vue.ref(0);
  const step = Vue.ref(1);
  const parity = Vue.computed(() =>
    count.value % 2 === 0 ? '偶数' : '奇数');
  const add = () => { count.value += step.value; };
  const reset = () => { count.value = 0; };
  return { count, step, parity, add, reset };
}
```

交付要求与自查表

- 交付：可运行的页面（或对照页）+ 一段差异说明
- 自查 8 步：
 - 1. 两个计数器点一下都能 +1?
 - 2. 原生版是否用了 `getElementById` / `textContent` / `addEventListener`?
 - 3. Vue 版是否用了 `createApp` + `setup` + `ref(0)`?
 - 4. 脚本里是 `count.value`，模板里是 `{{ count }}`?
 - 5. `script` 是否写在挂载点之后？控制台有没有报错？
 - 6. 能不能一句话说清命令式与声明式的区别？
 - 7. 能不能说清"为什么改了 `count.value` 页面就变了"？
 - 8. 加"减 1"时，两种方案各改了几处？

常见翻车点

- ① script 写在 #app 之前 → 取不到元素，mount 失败
- ② CDN 地址写错或断网 → Vue is not defined
- ③ querySelector 拿到 null 还继续用 → Cannot read property
- ④ ref 忘了写 .value → 数字永远不变
- ⑤ v-for 忘了写 :key → 列表更新错乱
- ⑥ v-if 与 v-show 混用分不清 → 该销毁的不销毁
- ⑦ textContent 写成 innerHTML → XSS 风险
- ⑧ 一个页面两个 Vue 实例用同一个 id → 只挂载第一个